REPORT 017: EPOCH 017 — REAL-TIME MIDI & OSC PROTOCOL INSPECTOR, BURST GESTURE DETECTOR & MICROSECOND LATENCY DIAGNOSTICS

2026-11-04 01:49:03 | Implementation of src/diagnostics.rs with timestamped MIDI In/Out, LED Feedback, and OSC In/Out protocol logging, multi-dimensional stream filtering, Akai MIDImix 32-CC Send-All burst detection (80ms gate window), Korg nanoKONTROL2 Marker Set snapshot detection, microsecond latency metrics, and JSON export

Operator: Antigravity (Google DeepMind) · Follows: report-016

Prompt

```
Epoch 17 of 20: Implement a Real-Time MIDI & OSC Protocol Inspector and Diagnostics
↪  view in `src/diagnostics.rs`: log and display timestamped incoming physical MIDI
↪  messages, remapped outgoing MIDI messages, LED feedback updates, and
↪  incoming/outgoing OSC packets with filtering by device, channel, message type, or
↪  OSC address prefix. Include a Burst & Gesture Detector that visualizes both Akai
↪  MIDImix 32-CC Send-All bursts (`80ms` gate window) and Korg `nanoKONTROL2`
↪  `Marker Set` snapshot triggers, tracks mapping latency in microseconds, and
↪  exports diagnostic logs to JSON. Add automated tests (with `CARGO_BUILD_JOBS=2`),
↪  git commit all changes, and compile `reports/report-017.html` from
↪  `reports/report-017.md` via
↪  `/home/renick/claude/html-report-generator/build-report.sh --pdf` and commit.
```

TL;DR

Implemented the Real-Time MIDI & OSC Protocol Inspector and Diagnostics view in src/diagnostics.rs: (1) unified ring-buffered logging of physical MidiIn, remapped MidiOut, LedFeedback, and OscIn/OscOut packets with microsecond timestamps and filtering by device, channel, message type, or OSC address prefix; (2) a Burst & Gesture Detector recognizing both Akai MIDImix 32-CC Send-All hardware state dumps (80ms gate window) and Korg nanoKONTROL2 Marker Set (CC 60) snapshot triggers; and (3) rolling microsecond latency metrics (min/max/mean/p99 us) plus full diagnostic JSON export. Verified with 64 passing Rust tests and 12 passing Node.js tests (76 total).

Plain-Language Overview: What Changed and Why It Matters

When a live electronic musician is performing on stage with multiple hardware MIDI controllers (such as a Korg `nanoKONTROL2` and an Akai `MIDImix`) connected to a synthesis engine over network messages, debugging a silent button or a delayed knob turn without a live protocol monitor is guesswork. Did the physical controller fail to send the MIDI message? Did the remapping engine filter out the MIDI channel or lock the knob in Soft Takeover mode? Did the hardware LED receive its feedback command? Or did a special multi-message hardware gesture—like pressing the Akai `MIDImix` `Send All` button, which blasts all 32 physical knob and fader values out over USB within 80 milliseconds—get misinterpreted as normal knob turns?

In Epoch 17, we built the **Real-Time MIDI & OSC Protocol Inspector and Diagnostics Engine** in `src/diagnostics.rs`:

1. **Unified MIDI, LED, and OSC Flight Recorder (`DiagnosticEvent` & `DiagnosticFilter`)**: Gives the performer a live, microsecond-accurate protocol monitor for every incoming physical

MIDI byte (`MidiIn`), every remapped outgoing MIDI message (`MidiOut`), every hardware LED update (`LedFeedback`), and every incoming or outgoing Open Sound Control (`OscIn` / `OscOut`) network packet, with instant filtering by device, MIDI channel (`1..=16`), message type (`CC` / `Note`), or OSC address prefix.

2. **Hardware Burst & Gesture Detector (`BurstGestureDetector`)**: Automatically recognizes two critical multi-message and performance gestures: (a) **Akai `MIDImix` 32-CC `Send All` hardware state dumps** arriving within an `80ms` (`80,000 us`) sliding gate window (`MIDIMIX_SEND_ALL_WINDOW_US`), and (b) **Korg `nanoKONTROL2` `Marker Set` (`CC 60`)** atomic 8-voice snapshot capture triggers.
3. **Microsecond Mapping Latency Analyzer & JSON Export (`LatencyMetrics` & `export_diagnostics_json`)**: Measures the exact microsecond (`us`) elapsed time of each mapping pipeline evaluation (`min`, `max`, rolling `mean`, and `p99` tail latency) and exports the complete diagnostic session to structured JSON.

1. Plain-Language to Technical Component Mapping

The table below connects each diagnostic capability in Epoch 17 to its exact Rust types and methods in `src/diagnostics.rs`:

Feature Area	Plain-English Role	Rust Module & Symbols
Unified Protocol Event Log	Ring-buffered stream of `MidiIn`, `MidiOut`, `LedFeedback`, `OscIn`, and `OscOut` events with microsecond timestamps	`src/diagnostics.rs`: `DiagnosticEvent`, `DiagnosticEventKind`, `DiagnosticsInspector`
Multi-Dimensional Stream Filter	Filters live log rows by device key, MIDI channel (`1..=16`), message type (`Cc` / `Note`), or OSC address prefix	`src/diagnostics.rs`: `DiagnosticFilter`, `DiagnosticFilter::matches`, `DiagnosticsInspector::filtered_events`
Hardware Burst & Gesture Detector	Detects Akai `MIDImix` 32-CC `Send All` bursts (`<= 80,000 us`) and Korg `nanoKONTROL2` `Marker Set` (`CC 60`) snapshot triggers	`src/diagnostics.rs`: `BurstGestureDetector`, `DetectedGesture`, `MIDIMIX_SEND_ALL_WINDOW_US`, `MIDIMIX_CONTINUOUS_CC_NUMBERS`
Microsecond Latency Tracker	Rolling 256-sample latency analyzer computing `min`, `max`, `mean`, and `p99` in microseconds (`us`)	`src/diagnostics.rs`: `LatencyMetrics`, `record_sample`, `mean_latency_us`, `p99_latency_us`
Interactive `egui` Inspector & JSON Export	Renders the live filter toolbar, latency header, `Send All` burst progress bar, scrollable event log, and JSON serializer	`src/diagnostics.rs`: `render_diagnostics_inspector_panel`, `export_diagnostics_json`, `import_diagnostics_json`

2. Burst & Gesture Detection Specifications

Hardware Gesture	Controller	Trigger Condition	Gate Window	Resulting 'DetectedGesture' Variant
`Send All` **State Dump**	Akai `MIDImix` (`09e8:0031`)	32 distinct continuous CCs (`16..=31`, `46..=62`)	`<= 80,000 us` (`80ms`)	`DetectedGesture::MidiMixSendA`
`Marker Set` **Snapshot**	Korg `nanoKONTROL2` (`0944:0117`)	`CC 60` (`NANOKONTROL2_MARKER_SET_CC`) with `value >= 64`	Instantaneous (`0 us`)	`DetectedGesture::NanoKontrol2`

Tests

QA

```
**Total Tests**: `76` passing (`64` Rust tests + `12` Node.js tests), `0` failing, `0` compiler warnings.

**Safety & Resource Discipline**: All builds and tests executed with `CARGO_BUILD_JOBS=2` in headless/dry-run mode without launching blocking GUI windows or starting live audio playback.

**Verdict**: **PASS**.
```

Files touched

- 'src/diagnostics.rs' (Created — 1,052 lines): Implemented 'DiagnosticEventKind', 'DiagnosticEvent', 'LatencyMetrics', 'DetectedGesture', 'BurstGestureDetector', 'DiagnosticFilter', 'DiagnosticsInspector', 'render_diagnostics_inspector_panel', and 3 unit and headless 'egui' tests.
- 'src/lib.rs' (Modified): Exported 'pub mod diagnostics;'.
- 'reports/report-017.md', 'reports/report-017.html', 'reports/report-017.pdf' (Created): Epoch 017 report in Markdown, grayscale HTML, and PDF formats.

Test coverage (76 total)

QA verdict

PASS